算法模板

Contents

赛前准备 2
- vscode 设置 . . . 2
- 常用头文件、宏定义等 . . . 2
- 易错注意 . . . 3

基本算法 3
- 快读 . . . 3
- 二分 . . . 4
- 三分 . . . 5

数学 5
- 快速幂 . . . 5
- 数论 . . . 5
  - 判断质数试除法 . . . 5
  - 欧拉筛 . . . 6
  - 分解质因数 . . . 6
  - 最小公倍数 . . . 6
  - 拓展欧几里得 . . . 7
  - 模逆元 . . . 7
- 高精度 . . . 8
  - 存储 . . . 8
  - 输出 . . . 8
  - 加法 . . . 8
  - 减法 . . . 9
  - 乘法 . . . 9
  - 除法 . . . 10
- 弧度制与坐标系 . . . 11
- 组合数学 . . . 11
  - 排列数 . . . 11
  - 组合数 . . . 12
  - 求数组中三元组乘积和 . . . 12
- 线性代数 . . . 12
  - 矩阵 . . . 12
  - 线性基 . . . 13

数据结构 15
- 并查集 . . . 15
- ST 表 . . . 16
- 树状数组 . . . 17
- 线段树 . . . 18

图论 25

Floyd 求传递闭包 . . . 25
拓扑排序 . . . 25
Kahn 算法 . . . 25
强连通分量（scc）缩点 . . . 25
Kosaraju . . . 25
树 . . . 28
LCA . . . 28
计算每个节点作为根时，子节点的子树的大小（CF2241E） . . . 30
网络流 . . . 30
Dinic . . . 30
费用流 . . . 31

字符串 33
KMP . . . 33
Z 函数 . . . 34
Manacher . . . 34
字典树 . . . 35
AC 自动机 . . . 35

动态规划 37
线性 DP . . . 37
最长上升子序列 . . . 37
状压 DP . . . 38
状压 DP 求拓扑序数量 . . . 38

赛前准备

vscode 设置

开启 autosave

终端编译运行（如无 cph）：

```
g++ a.cpp -o a -std = c++17 -O2 -Wall
./a < 1.in > 1.out
```

脚本：

```
#!/usr/bin/env bash
# 用法: ./run.sh [a.cpp] [1.in] [1.out]
src=${1:-a.cpp}
in=${2:-1.in}
out=${3:-1.out}

g++ "$src" -o "${src%.cpp}" -std=c++17 -O2 -Wall || exit 1
./"${src%.cpp}" < "$in" > "$out"
echo " 退出码: $?"
```

保存为 run.sh，chmod +x run.sh，然后./run.sh 即可。默认 a.cpp → 1.in → 1.out，也可以传参换文件。

常用头文件、宏定义等

```
#include <bits/stdc++.h>
using namespace std;
#define endl '\n'
#define rep(i,a,b) for(int i=a;i<=b;i++)
#define repd(i,a,b) for(int i=a;i>=b;i--)
```

```cpp
#define PII pair<int,int>
#define int long long
#define dbg(a) cout << #a << "=" << a << endl;

void solve(){

    return;
}

signed main(){
    ios::sync_with_stdio(0),cin.tie(0),cout.tie(0);
    int _;cin >> _;
    while(_--)
    solve();
    return 0;
}
```

易错注意

判断平方数精度

`if(a[i]==(int)sqrt(a[i])*(int)sqrt(a[i]))`

位运算要加括号!!!（!? 加加?!）

`a[i]<(a[i]^xor_sum)`

右移左边的 1 记得加 LL

`(1LL<<40)`

不要用模意义下的数比较大小

longlong 和 size_t 比较时会被转化成 size_t ， 如果是负数会溢出

基本算法

快读

神秘输入 1e7 卡常时使用

```cpp
class FastScanner {
    static const int BUFSIZE = 1 << 20;  // 1MB 缓冲区
    char buf[BUFSIZE];
    int idx, size;

public:
    FastScanner() : idx(0), size(0) {}

    // 读取单个字符（跳过空白字符）
    char readChar() {
        char c = getChar();
        while (c <= ' ') c = getChar();  // 跳过空格、换行等
        return c;
    }

    // 读取整数
    int readInt() {
```

```
        char c = getChar();
        int sign = 1, val = 0;
        while (c <= ' ') c = getChar();  // 跳过空白
        if (c == '-') {
            sign = -1;
            c = getChar();
        }
        while (c >= '0' && c <= '9') {
            val = val * 10 + (c - '0');
            c = getChar();
        }
        return val * sign;
    }

    // 读取字符串（以空白字符结尾）
    void readString(char* s) {
        char c = getChar();
        while (c <= ' ') c = getChar();
        while (c > ' ') {
            *s++ = c;
            c = getChar();
        }
        *s = '\0';
    }

private:
    // 从缓冲区获取下一个字符
    char getChar() {
        if (idx >= size) {
            size = fread(buf, 1, BUFSIZE, stdin);
            idx = 0;
            if (size == 0) return 0;  // EOF
        }
        return buf[idx++];
    }
};
```

二分

```
// 值域 [L, R] 找最小合法 x
int get_min(int L, int R) {
    while (L < R) {
        int mid = L + (R - L) / 2;
        if (check(mid))
            R = mid;
        else
            L = mid + 1;
    }
    return check(L) ? L : -1;
}
// 值域 [L, R] 找最大合法 x
int get_max(int L, int R) {
    while (L < R) {
        int mid = L + (R - L + 1) / 2; // 向上取整
```

```
        if (check(mid))
            L = mid;
        else
            R = mid - 1;
    }
    return check(L) ? L : -1;
}
```

三分

```
// 对于整数域上的凸函数，在 [l, r] 内求极小值点
int ternary_search_int_min(int l, int r) {
    while (r - l > 2) {
        int m1 = l + (r - l) / 3;
        int m2 = r - (r - l) / 3;

        if (f(m1) < f(m2)) {
            r = m2;
        } else {
            l = m1;
        }
    }
    // 在剩余区间 [l, r] 中枚举找最优
    int ans = l;
    for (int i = l + 1; i <= r; i++) {
        if (f(i) < f(ans)) ans = i;
    }
    return ans;
}
```

数学

快速幂

```
int ksm(int a,int b){
    int res=1;
    while(b){
        if(b&1) res=res*a%mod;
        b=b>>1;
        a=a*a%mod;
    }
    return res;
}
```

数论

判断质数试除法

```
bool isPrime(int a) {
    if (a < 2) return false;
    for (int i = 2; i*i <= a; ++i)
        if (a % i == 0) return false;
    return true;
}
```

欧拉筛

```
vector<int> prime;
void p(int n){
    vector<int> a(n+1);
    for(int i=2;i<=n;i++){
        if(a[i]==0){
            prime.push_back(i);
        }
        for(auto it:prime){
            if(it>n/i) break;
            a[i*it]=1;
            if(i%it==0) break;
        }
    }
}
```

分解质因数

```
vector<int> breakdown(int N) {
    vector<int> result;
    for (int i = 2; i * i <= N; i++) {
        if (N % i == 0) {  // 如果 i 能够整除 N，说明 i 为 N 的一个质因子.
            while (N % i == 0) N /= i;
            result.push_back(i);
        }
    }
    if (N != 1) {  // 说明再经过操作之后 N 留下了一个素数
        result.push_back(N);
    }
    return result;
}
// 分解质因数，返回 (素数，指数)
vector<pair<int, int>> factor(int x) {
    vector<pair<int, int>> res;
    for (int i = 2; i * i <= x; i++) {
        if (x % i == 0) {
            int cnt = 0;
            while (x % i == 0) cnt++, x /= i;
            res.emplace_back(i, cnt);
        }
    }
    if (x > 1) res.emplace_back(x, 1);
    return res;
}
```

最小公倍数

```
vector<int> primes;
vector<bool> vis;

void euler(int n) {
    vis.assign(n + 1, false);
    primes.clear();
    for (int i = 2; i <= n; i++) {
```

```
        if (!vis[i]) primes.push_back(i);
        for (int p : primes) {
            if (1LL * i * p > n) break;
            vis[i * p] = true;
            if (i % p == 0) break;
        }
    }
}

int LCM_mod_fast(int n, int mod) {
    if (mod == 1) return 0;
    euler(n);
    int ans = 1;
    for (int p : primes) {
        int cnt = 0, now = 1;
        while (1LL * now * p <= n) {
            now *= p;
            cnt++;
        }
        ans = ans * qpow(p, cnt, mod) % mod;
    }
    return ans;
}
```

拓展欧几里得

```
int Exgcd(int a, int b, int &x, int &y) {
      if (!b) {
        x = 1;
        y = 0;
        return a;
      }
      int d = Exgcd(b, a % b, x, y);
      int t = x;
      x = y;
      y = t - (a / b) * y;
      return d;
}
```

模逆元

```
int inverse(int a, int p) { return pow(a, p - 2, p); }
```

预处理阶乘逆元

```
int inv[MAXN];
int fac[MAXN];
int invfac[MAXN];
void pre_all(int n)
{
    fac[0] = 1;
    for(int i=1;i<=n;i++) fac[i]=fac[i-1]*i%MOD;
    inv[1]=1;
    for(int i=2;i<=n;i++) inv[i]=MOD-MOD/i*inv[MOD%i]%MOD;
    invfac[n]=qpow(fac[n],MOD-2);
```

```
    for(int i=n-1;i>=0;i--) invfac[i]=invfac[i+1]*(i+1)%MOD;
}
```

高精度

存储

```
void clear(int a[]) {
    for (int i = 0; i < LEN; ++i) a[i] = 0;
}

void read(int a[]) {
    static char s[LEN + 1];
    scanf("%s", s);

    clear(a);

    int len = strlen(s);
    // 如上所述，反转
    for (int i = 0; i < len; ++i) a[len - i - 1] = s[i] - '0';
    // s[i] - '0' 就是 s[i] 所表示的数码
    // 有些同学可能更习惯用 ord(s[i]) - ord('0') 的方式理解
}
```

输出

```
void print(int a[]) {
    int i;
    for (i = LEN - 1; i >= 1; --i)
        if (a[i] != 0) break;
    for (; i >= 0; --i) putchar(a[i] + '0');
    putchar('\n');
}
```

加法

```
void add(int a[], int b[], int c[]) {
    clear(c);

    // 高精度实现中，一般令数组的最大长度 LEN 比可能的输入大一些
    // 然后略去末尾的几次循环，这样一来可以省去不少边界情况的处理
    // 因为实际输入不会超过 1000 位，故在此循环到 LEN - 1 = 1003 已经足够
    for (int i = 0; i < LEN - 1; ++i) {
        // 将相应位上的数码相加
        c[i] += a[i] + b[i];
        if (c[i] >= 10) {
            // 进位
            c[i + 1] += 1;
            c[i] -= 10;
        }
    }
}
```

减法

```
void sub(int a[], int b[], int c[]) {
    clear(c);

    for (int i = 0; i < LEN - 1; ++i) {
        // 逐位相减
        c[i] += a[i] - b[i];
        if (c[i] < 0) {
            // 借位
            c[i + 1] -= 1;
            c[i] += 10;
        }
    }
}
```

事实上，上面的代码只能处理减数　　大于等于被减数　　的情况．处理被减数比减数小，即　　<　　时的情况很简单．

　　=　（　　）

要计算　　的值，因为有　　>　，可以调用以上代码中的 sub 函数，写法为 sub(b,a,c)．要得到　　的值，在得数前加上负号即可．

乘法

高精度 * 低精度

```
void mul_short(int a[], int b, int c[]) {
    clear(c);

    for (int i = 0; i < LEN - 1; ++i) {
        // 直接把 a 的第 i 位数码乘以乘数，加入结果
        c[i] += a[i] * b;

        if (c[i] >= 10) {
            // 处理进位
            // c[i] / 10 即除法的商数成为进位的增量值
            c[i + 1] += c[i] / 10;
            // 而 c[i] % 10 即除法的余数成为在当前位留下的值
            c[i] %= 10;
        }
    }
}
```

高精度 * 高精度

```
void mul(int a[], int b[], int c[]) {
    clear(c);

    for (int i = 0; i < LEN - 1; ++i) {
        // 这里直接计算结果中的从低到高第 i 位，且一并处理了进位
        // 第 i 次循环为 c[i] 加上了所有满足 p + q = i 的 a[p] 与 b[q] 的乘积之和
        // 这样做的效果和直接进行上图的运算最后求和是一样的，只是更加简短的一种实现方式
        for (int j = 0; j <= i; ++j) c[i] += a[j] * b[i - j];

        if (c[i] >= 10) {
            c[i + 1] += c[i] / 10;
```

```
            c[i] %= 10;
        }
    }
}
```

除法

为了减少冗余运算，我们提前得到被除数的长度　　与除数的长度　　!，从下标　　开始，从高位到低位来计算商．这和手工计算时将第一次乘法的最高位与被除数最高位对齐的做法是一样的．

参考程序实现了一个函数 `greater_eq()` 用于判断被除数以下标 `last_dg` 为最低位，是否可以再减去除数而保持非负．此后对于商的每一位，不断调用 `greater_eq()`，并在成立的时候用高精度减法从余数中减去除数，也即模拟了竖式除法的过程．

```
// 被除数 a 以下标 last_dg 为最低位，是否可以再减去除数 b 而保持非负
// len 是除数 b 的长度，避免反复计算
bool greater_eq(int a[], int b[], int last_dg, int len) {
    // 有可能被除数剩余的部分比除数长，这个情况下最多多出 1 位，故如此判断即可
    if (a[last_dg + len] != 0) return true;
    // 从高位到低位，逐位比较
    for (int i = len - 1; i >= 0; --i) {
        if (a[last_dg + i] > b[i]) return true;
        if (a[last_dg + i] < b[i]) return false;
    }
    // 相等的情形下也是可行的
    return true;
}

void div(int a[], int b[], int c[], int d[]) {
    clear(c);
    clear(d);

    int la, lb;
    for (la = LEN - 1; la > 0; --la)
        if (a[la - 1] != 0) break;
    for (lb = LEN - 1; lb > 0; --lb)
        if (b[lb - 1] != 0) break;
    if (lb == 0) {  // 除数不能为零
        puts("> <");
        return;
    }

    // c 是商
    // d 是被除数的剩余部分，算法结束后自然成为余数
    for (int i = 0; i < la; ++i) d[i] = a[i];
    for (int i = la - lb; i >= 0; --i) {
        // 计算商的第 i 位
        while (greater_eq(d, b, i, lb)) {
            // 若可以减，则减
            // 这一段是一个高精度减法
            for (int j = 0; j < lb; ++j) {
                d[i + j] -= b[j];
                if (d[i + j] < 0) {
                    d[i + j + 1] -= 1;
                    d[i + j] += 10;
```

```
                }
            }
            // 使商的这一位增加 1
            c[i] += 1;
            // 返回循环开头，重新检查
        }
    }
}
```

弧度制与坐标系

在 C/C++ 语言中，一般取 为 acos(-1)

=atan2(,)!. 注意上述函数的值域为 (,].

在 C/C++ 语言的 <math.h> 或 <cmath> 库里定义了调用 atan2(y, x) 即可.

极角排序

```
struct Point {
    long long x, y;
};

long long cross(const Point& a, const Point& b) { return a.x * b.y - a.y * b.x; }
long long dot  (const Point& a, const Point& b) { return a.x * b.x + a.y * b.y; }

// 半平面划分：以负 x 轴为间断点切开
// 0 -> 角度 [0, )   1 -> 角度 [ , 2 )
inline int half(const Point& p) {
    return (p.y > 0 || (p.y == 0 && p.x > 0)) ? 0 : 1;
}

// 极角排序（以原点为极点，逆时针，起点为正 x 轴）
bool polar_cmp(const Point& a, const Point& b) {
    int ha = half(a), hb = half(b);
    if (ha != hb) return ha < hb;       // 不同半平面直接分先后
    long long c = cross(a, b);
    if (c != 0) return c > 0;           // 同半平面内用叉积定逆时针
    return dot(a, a) < dot(b, b);       // 共线同向：按距离由近到远
}
```

组合数学

排列数

```
const int mod=998244353;
const int MAXN=2e5+10;
int fac[MAXN];
int ksm(int a,int b){
    int res=1;
    while(b){
        if(b&1) res=res*a%mod;
        b>>=1;
        a=a*a%mod;
    }
    return res;
}
```

```
void pre_all(int n)
{
    fac[0] = 1;
    for(int i=1;i<=n;i++) fac[i]=fac[i-1]*i%mod;
}
int A(int n,int m,int p){
    return fac[n]*ksm(n-m,p-2)%p;
}
```

组合数

```
const int mod=998244353;
const int MAXN=2e5+10;
int fac[MAXN];
int ksm(int a,int b){
    int res=1;
    while(b){
        if(b&1) res=res*a%mod;
        b>>=1;
        a=a*a%mod;
    }
    return res;
}
void pre_all(int n)
{
    fac[0] = 1;
    for(int i=1;i<=n;i++) fac[i]=fac[i-1]*i%mod;
}

int C(int n,int m,int p){
    return fac[n]*ksm(fac[m]*fac[n-m]%p,p-2)%p;
}
```

求数组中三元组乘积和

```
int calcu(const vector<int>& a){
    if(a.size()<3) return 0;
    int s1=0,s2=0,s3=0;
    for(auto x:a){
        s1+=x;
        s2+=x*x;
        s3+=x*x*x;
    }
    return (s1*s1*s1-3*s1*s2+2*s3)/6;
}
```

线性代数

矩阵

```
struct mat {
    int a[sz][sz];

    mat() { memset(a, 0, sizeof a); }
```

```
    mat operator-(const mat& T) const {
        mat res;
        for (int i = 0; i < sz; ++i)
            for (int j = 0; j < sz; ++j) {
                res.a[i][j] = (a[i][j] - T.a[i][j]) % MOD;
            }
        return res;
    }

    mat operator+(const mat& T) const {
        mat res;
        for (int i = 0; i < sz; ++i)
            for (int j = 0; j < sz; ++j) {
                res.a[i][j] = (a[i][j] + T.a[i][j]) % MOD;
            }
        return res;
    }

    mat operator*(const mat& T) const {
        mat res;
        int r;
        for (int i = 0; i < sz; ++i)
            for (int k = 0; k < sz; ++k) {
                r = a[i][k];
                for (int j = 0; j < sz; ++j)
                    res.a[i][j] += T.a[k][j] * r, res.a[i][j] %= MOD;
            }
        return res;
    }

    mat operator^(int x) const {
        mat res, bas;
        for (int i = 0; i < sz; ++i) res.a[i][i] = 1;
        for (int i = 0; i < sz; ++i)
            for (int j = 0; j < sz; ++j) bas.a[i][j] = a[i][j] % MOD;
        while (x) {
            if (x & 1) res = res * bas;
            bas = bas * bas;
            x >>= 1;
        }
        return res;
    }
};
```

线性基

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

// 极简全能线性基（高斯消元版）
struct LinearBasis {
    int a[61], tmp[61];
```

```cpp
bool zero = 0;
int cnt = 0;

void init(){
    memset(a,0,sizeof a);
    memset(tmp,0,sizeof tmp);
}

// 插入一个数
void insert(int x) {
    for (int i = 60; ~i; --i) {
        if (x >> i & 1) {
            if (!a[i]) { a[i] = x; break; }
            x ^= a[i];
        }
    }
    if (!x) zero = 1;
}

// 求最大异或和
int qmax() {
    int res = 0;
    for (int i = 60; ~i; --i)
        if ((res ^ a[i]) > res) res ^= a[i];
    return res;
}

// 求最小异或和
int qmin() {
    if (zero) return 0;
    for (int i = 0; i <= 60; ++i)
        if (a[i]) return a[i];
    return 0;
}

// 重构（求第 k 小必须用）
void rebuild() {
    cnt = 0;
    fill(tmp, tmp + 61, 0);
    for (int i = 0; i <= 60; ++i) {
        for (int j = i - 1; ~j; --j)
            if (a[i] >> j & 1) a[i] ^= a[j];
        if (a[i]) tmp[cnt++] = a[i];
    }
}

// 求第 k 小异或和
int qkth(int k) {
    if (zero) k--;
    if (k >= (1LL << cnt)) return -1;
    int res = 0;
    for (int i = 0; i < cnt; ++i)
        if (k >> i & 1) res ^= tmp[i];
    return res;
```

```
    }

    // 能否异或出 x
    bool have(int x) {
        for (int i = 60; ~i; --i)
            if (x >> i & 1) x ^= a[i];
        return x == 0;
    }
} lb; // 全局变量，直接用

int main() {
    int n;
    cin >> n;
    // 1. 把所有数插入线性基
    for (int i = 0; i < n; ++i) {
        int x; cin >> x;
        lb.insert(x);
    }

    // 2. 求最大
    cout << " 最大: " << lb.qmax() << endl;

    // 3. 求最小
    cout << " 最小: " << lb.qmin() << endl;

    // 4. 求第 k 小（必须先重构）
    lb.rebuild();
    cout << " 第 2 小: " << lb.qkth(2) << endl;

    // 5. 能否异或出 3
    cout << " 能否出 3: " << lb.have(3) << endl;
}
```

lb.insert(x) ——把数丢进去

lb.qmax() ——最大异或和

lb.qmin() ——最小异或和

lb.rebuild() → lb.qkth(k) ——第 k 小

lb.have(x) ——能不能异或出 x

求第 k 小必须先调用 rebuild()

能异或出 0 时，最小值就是 0

所有数支持到 1e18（64 位）

数据结构

并查集

```
int parent[MAX];
int find(int x){
    if(parent[x]!=x) parent[x]=find(parent[x]);
    return parent[x];
}
```

使用 parent[find（x)]=find(y) 进行合并

ST 表

```
const int N = 1e5 + 10;
const int LOG = 20; // log2(1e5)  17，开 20 足够

int a[N];
int st_max[LOG][N]; // st_max[k][i]：从 i 开始，长度为 2^k 的区间最大值
int st_min[LOG][N]; // 区间最小值
int log2n[N];       // 预处理 log2，加速查询

// 预处理 log2 数组
void pre_log(int n)
{
    log2n[1] = 0;
    for (int i = 2; i <= n; i++)
        log2n[i] = log2n[i / 2] + 1;
}

// 构建 ST 表
void build(int n)
{
    // 初始化 k=0，区间长度 2^0 = 1
    for (int i = 1; i <= n; i++)
    {
        st_max[0][i] = a[i];
        st_min[0][i] = a[i];
    }

    // 倍增递推
    for (int k = 1; k < LOG; k++)
    {
        for (int i = 1; i + (1 << k) - 1 <= n; i++)
        {
            st_max[k][i] = max(st_max[k-1][i], st_max[k-1][i + (1 << (k-1))]);
            st_min[k][i] = min(st_min[k-1][i], st_min[k-1][i + (1 << (k-1))]);
        }
    }
}

// 查询 [l, r] 最大值
int query_max(int l, int r)
{
    int len = r - l + 1;
    int k = log2n[len];
    return max(st_max[k][l], st_max[k][r - (1 << k) + 1]);
}

// 查询 [l, r] 最小值
int query_min(int l, int r)
{
    int len = r - l + 1;
    int k = log2n[len];
```

```
    return min(st_min[k][l], st_min[k][r - (1 << k) + 1]);
}
```

树状数组

```
int getsum(int x) {  // a[1]..a[x] 的和
    int ans = 0;
    while (x > 0) {
        ans = ans + c[x];
        x = x - lowbit(x);
    }
    return ans;
}

void add(int x, int k) {
    while (x <= n) {  // 不能越界
        c[x] = c[x] + k;
        x = x + lowbit(x);
    }
}
```

O(n) 建树

```
void init() {
    for (int i = 1; i <= n; ++i) {
        t[i] += a[i];
        int j = i + lowbit(i);
        if (j <= n) t[j] += t[i];
    }
}
```

如果需要多个可用封装好的结构体

```
struct FenwickTree {
    vector<int> tr;
    int n;
    int lowbit(int x) {
        return x & -x;
    }
    // 初始化大小
    void init(int size) {
        n = size;
        tr.resize(n+1);
        for (int i = 0; i <= n; ++i) tr[i] = 0;
    }
    // 单点 x 加 val
    void add(int x, int val) {
        for (; x <= n; x += lowbit(x))
            tr[x] += val;
    }
    // 查询 [1, x] 前缀和
    int query(int x) {
        int res = 0;
        for (; x; x -= lowbit(x))
            res += tr[x];
        return res;
```

```
    }
    // 查询 [l, r] 区间和
    int range_query(int l, int r) {
        return query(r) - query(l - 1);
    }
};
```

区间最大值

```
int getmax(int l, int r) {
    int ans = 0;
    while (r >= l) {
        ans = max(ans, a[r]);
        --r;
        for (; r - lowbit(r) >= l; r -= lowbit(r)) {
            // 注意，循环条件不要写成 r - lowbit(r) + 1 >= l
            // 否则 l = 1 时，r 跳到 0 会死循环
            ans = max(ans, C[r]);
        }
    }
    return ans;
}
```

线段树

模板 1

```
//洛谷 P3372 线段树，区间修改 + 区间查询
#include <bits/stdc++.h>
using namespace std;
#define int long long
const int N = 1e5 + 10;
int a[N];

int tree[N << 2];    //tree[i] 为第 i 个节点的值，表示一个线段区间的值，如最值、区间和
int tag[N << 2];     //tag[] 为第 i 个节点的 Lazy-Tag，记录这个区间的修改值

int ls(int p) { return p << 1; }     //左儿子 p*2
int rs(int p) { return p << 1 | 1; } //右儿子 p*2+1

void push_up(int p){                          //从下向上传递区间值

    //
    tree[p] = tree[ls(p)] + tree[rs(p)];
    //

    //求最值区间和，如果是求最小值改为 tree[p] = min(tree[ls(p)], tree[rs(p)]);
}

void build(int p, int pl, int pr){            //建树。p 为节点编号，它指向区间 [pl, pr]
    tag[p] = 0;                               //Lazy-Tag 标记初始化
    if(pl == pr){tree[p] = a[pl]; return;}    //最底层的叶子节点，赋值
    int mid = (pl + pr) >> 1;                 //分治：折半
    build(ls(p), pl, mid);                    //左儿子
    build(rs(p), mid + 1, pr);                //右儿子
    push_up(p);                               //从下往上传递区间值
```

```
}

void addtag(int p, int pl, int pr, int d){     //给节点 p 打 tag 标记，并更新 tree
    tag[p] += d;                               //打上 tag 标记

    //
    tree[p] += d * (pr - pl + 1);              //计算新的 tree 值
    //

}

void push_down(int p, int pl, int pr){         //不能覆盖时，把 tag 传给子树
    if(tag[p]){                                //有 tag 标记，这是以前做区间修改时留下的
        int mid = (pl + pr) >> 1;
        addtag(ls(p), pl, mid, tag[p]);        //把 tag 标记传给左子树
        addtag(rs(p), mid + 1, pr, tag[p]);    //把 tag 标记传给右子树
        tag[p] = 0;                            //p 自己的 tag 被传走了，归零
    }
}

void update(int L, int R, int p, int pl, int pr, int d){ //区间修改：[L,R] 内每个元素加 d
    if(L <= pl && pr <= R){                    //完全覆盖，直接返回这个节点，它的子树不用再深入了
        addtag(p, pl, pr, d);                  //给节点 p 打 tag 标记，下一次区间修改到 p 时会用到
        return;
    }
    push_down(p, pl, pr);                      //如果不能覆盖，把 tag 传给子树
    int mid = (pl + pr) >> 1;
    if(L <= mid) update(L, R, ls(p), pl, mid, d);     //递归左子树
    if(R > mid)  update(L, R, rs(p), mid + 1, pr, d);//递归右子树
    push_up(p);
}

int query(int L, int R, int p, int pl, int pr){
    //查询区间 [L,R],p 是当前节点（线段）的编号,[pl,pr] 是节点 p 表示的线段区间
    if(L <= pl && pr <= R) return tree[p];     //完全覆盖，直接返回
    push_down(p, pl, pr);                      //不能覆盖，递归子树

    //
    int res = 0;
    //

    int mid = (pl + pr) >> 1;

    //
    if(L <= mid) res += query(L, R, ls(p), pl, mid);   //左子节点有重叠
    if(R > mid)  res += query(L, R, rs(p), mid + 1, pr);//右子节点有重叠
    //

    return res;
}

int main(){
    int n, m;
    scanf("%LLd%LLd", &n, &m);
```

```
    for(int i = 1; i <= n; i++) scanf("%LLd", &a[i]);
    build(1, 1, n); //建树

    while(m--){
        int q, L, R, d;
        scanf("%LLd", &q);
        if(q == 1){                         //区间修改：[L,R] 的每个元素加 d
            scanf("%LLd%LLd%LLd", &L, &R, &d);
            update(L, R, 1, 1, n, d);
        }else{                              //区间查询：[L,R] 区间和
            scanf("%LLd%LLd", &L, &R);
            printf("%LLd\n", query(L, R, 1, 1, n));
        }
    }
    return 0;
}
```

模板 2

```
struct SegTree {
    int n;
    vector<int> tree;
    vector<int> lazy;

    SegTree(int size) {
        n = size;
        tree.resize(4 * n, 0);
        lazy.resize(4 * n, 0);
    }

    void pushup(int u) {
        tree[u] = tree[u << 1] + tree[u << 1 | 1];
    }

    // 下传懒标记
    void pushdown(int u, int l, int r) {
        if (lazy[u] == 0) return;
        int mid = (l + r) / 2;
        int left = u << 1;
        int right = u << 1 | 1;

        // 左子区间长度 mid-l+1
        tree[left] += lazy[u] * (mid - l + 1);
        lazy[left] += lazy[u];

        // 右子区间长度 r-mid
        tree[right] += lazy[u] * (r - mid);
        lazy[right] += lazy[u];

        lazy[u] = 0;
    }

    void build(int u, int l, int r, vector<int>& a) {
        if (l == r) {
            tree[u] = a[l];
```

```
            return;
        }
        int mid = (l + r) / 2;
        build(u<<1, l, mid, a);
        build(u<<1|1, mid+1, r, a);
        pushup(u);
    }

    // 区间 [L,R] 加上 val
    void range_add(int u, int l, int r, int L, int R, int val) {
        if (R < l || r < L) return;
        if (L <= l && r <= R) {
            tree[u] += val * (r - l + 1);
            lazy[u] += val;
            return;
        }
        pushdown(u, l, r);
        int mid = (l + r) / 2;
        range_add(u<<1, l, mid, L, R, val);
        range_add(u<<1|1, mid+1, r, L, R, val);
        pushup(u);
    }

    // 查询区间和
    int query(int u, int l, int r, int ql, int qr) {
        if (qr < l || r < ql) return 0;
        if (ql <= l && r <= qr) return tree[u];
        pushdown(u, l, r);
        int mid = (l + r) / 2;
        return query(u<<1,l,mid,ql,qr) + query(u<<1|1,mid+1,r,ql,qr);
    }

    void build(vector<int>& a) { build(1,1,n,a); }
    void range_add(int L, int R, int val) { range_add(1,1,n,L,R,val); }
    int query(int l, int r) { return query(1,1,n,l,r); }
};
```

值域线段树

```
// 静态堆式值域线段树：区间最大值
struct SegTree {
    int n;
    vector<int> tr;
    SegTree() {}
    SegTree(int sz) {
        n = 1;
        while(n < sz) n <<= 1;
        tr.assign(n * 2, 0);
    }
    void init(int sz) {
        n = 1;
        while(n < sz) n <<= 1;
        tr.assign(n * 2, 0);
    }
    void update(int pos, int val) {
```

```
        pos += n - 1;
        if(tr[pos] >= val) return;
        tr[pos] = val;
        for(pos >>= 1; pos >= 1; pos >>= 1) {
            int newv = max(tr[pos<<1], tr[pos<<1|1]);
            if(tr[pos] == newv) break;
            tr[pos] = newv;
        }
    }
    int query(int l, int r) {
        int res = 0;
        l += n - 1, r += n - 1;
        for(; l <= r; l >>= 1, r >>= 1) {
            if(l & 1) res = max(res, tr[l++]);
            if(!(r & 1)) res = max(res, tr[r--]);
        }
        return res;
    }
};
```

一个模板搞定三种线段树

```
// S : 值类型（求和场景可写成 {sum, len} 结构体）
// F : 懒标记类型
// op          : S × S -> S      合并两个值
// e           : () -> S         值的单位元
// mapping     : F × S -> S      把标记作用到值上
// composition: F × F -> F       标记叠加（先 g 后 f，即 f∘g）
// id          : () -> F         标记的单位元
template <class S, class F,
          S (*op)(S, S), S (*e)(),
          S (*mapping)(F, S), F (*composition)(F, F), F (*id)()>
struct SegTree {
    int n;
    vector<S> tree;
    vector<F> lazy;

    // 初始化：操作类型和单位元都在模板参数里确定
    SegTree(int size) {
        n = size;
        tree.assign(4 * n, e());
        lazy.assign(4 * n, id());
    }

    void pushup(int u) {
        tree[u] = op(tree[u << 1], tree[u << 1 | 1]);
    }

    // 下传懒标记
    void pushdown(int u, int l, int r) {
        if (lazy[u] == id()) return;
        int mid = (l + r) / 2;
        int left = u << 1;
        int right = u << 1 | 1;
```

```
        // 标记作用到子节点
        tree[left]  = mapping(lazy[u], tree[left]);
        tree[right] = mapping(lazy[u], tree[right]);
        lazy[left]  = composition(lazy[u], lazy[left]);
        lazy[right] = composition(lazy[u], lazy[right]);

        lazy[u] = id();
    }

    void build(int u, int l, int r, vector<S>& a) {
        if (l == r) {
            tree[u] = a[l];
            return;
        }
        int mid = (l + r) / 2;
        build(u<<1, l, mid, a);
        build(u<<1|1, mid+1, r, a);
        pushup(u);
    }

    // 区间 [L,R] 作用标记 val
    void range_add(int u, int l, int r, int L, int R, F val) {
        if (R < l || r < L) return;
        if (L <= l && r <= R) {
            tree[u] = mapping(val, tree[u]);
            lazy[u] = composition(val, lazy[u]);
            return;
        }
        pushdown(u, l, r);
        int mid = (l + r) / 2;
        range_add(u<<1, l, mid, L, R, val);
        range_add(u<<1|1, mid+1, r, L, R, val);
        pushup(u);
    }

    // 查询区间 [ql,qr]
    S query(int u, int l, int r, int ql, int qr) {
        if (qr < l || r < ql) return e();       // 空区间返回单位元
        if (ql <= l && r <= qr) return tree[u];
        pushdown(u, l, r);
        int mid = (l + r) / 2;
        S L = query(u<<1, l, mid, ql, qr);
        S R = query(u<<1|1, mid+1, r, ql, qr);
        return op(L, R);
    }

    void build(vector<S>& a) { build(1, 1, n, a); }
    void range_add(int L, int R, F val) { range_add(1, 1, n, L, R, val); }
    S query(int l, int r) { return query(1, 1, n, l, r); }
};
```

使用示例一：区间加 + 区间最值（S = F = long long）

```
using S = long long;
using F = long long;
```

```
S op_min(S a, S b) { return a < b ? a : b; }
S e_min()          { return (S)4e18; }

S op_max(S a, S b) { return a > b ? a : b; }
S e_max()          { return (S)-4e18; }

S mapping_add(F f, S x)    { return x + f; }
F composition_add(F f, F g){ return f + g; }
F id_add()                 { return 0; }

int main() {
    int n = 6;
    vector<S> a(n + 1);
    for (int i = 1; i <= n; i++) a[i] = i * 2 - 1;  // 1 3 5 7 9 11

    SegTree<S, F, op_min, e_min, mapping_add, composition_add, id_add> seg_min(n);
    SegTree<S, F, op_max, e_max, mapping_add, composition_add, id_add> seg_max(n);
    seg_min.build(a);
    seg_max.build(a);

    cout << seg_min.query(1, 5) << '\n';   // 1
    cout << seg_max.query(1, 5) << '\n';   // 9

    seg_min.range_add(1, 5, 10);           // 给 [1,5] 整体 +10
    cout << seg_min.query(2, 4) << '\n';   // 13
}
```

使用示例二：区间加 + 区间和（S 里带 len）

求和场景下 mapping 必须知道区间长度才能正确给和加上 f * len。ACL 的解法是把长度放进 S：

```
struct S { long long sum, len; };
using F = long long;

S op_sum(S a, S b)         { return {a.sum + b.sum, a.len + b.len}; }
S e_sum()                  { return {0, 0}; }
S mapping_add(F f, S x)    { return {x.sum + f * x.len, x.len}; }
F composition_add(F f, F g){ return f + g; }
F id_add()                 { return 0; }

int main() {
    int n = 6;
    vector<S> a(n + 1);
    for (int i = 1; i <= n; i++) a[i] = {i * 2 - 1, 1};   // 每项 len = 1

    SegTree<S, F, op_sum, e_sum, mapping_add, composition_add, id_add> seg(n);
    seg.build(a);

    cout << seg.query(1, 3).sum << '\n';   // 1+3+5 = 9

    seg.range_add(1, 3, 10);
    cout << seg.query(1, 3).sum << '\n';   // 39
}
```

图论

Floyd 求传递闭包

```
// std::bitset<SIZE> f[SIZE];
for (k = 1; k <= n; k++)
    for (i = 1; i <= n; i++)
        if (f[i][k]) f[i] = f[i] | f[k];
```

拓扑排序

Kahn 算法

```
int n, m;
vector<int> G[MAXN];
int in[MAXN];  // 存储每个结点的入度

bool toposort() {
    vector<int> L;
    queue<int> S;
    for (int i = 1; i <= n; i++)
        if (in[i] == 0) S.push(i);
    while (!S.empty()) {
        int u = S.front();
        S.pop();
        L.push_back(u);
        for (auto v : G[u]) {
            if (--in[v] == 0) {
                S.push(v);
            }
        }
    }
    if (L.size() == n) {
        for (auto i : L) cout << i << ' ';
        return true;
    }
    return false;
}
```

强连通分量（scc）缩点

Kosaraju

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <unordered_set>
using namespace std;

const int MAXN = 100005;

vector<int> g[MAXN];      // 原图 g[u]：u 的出边
vector<int> gr[MAXN];     // 反图 gr[u]：所有指向 u 的边，也就是原图 v->u 变成 gr[u]->v

bool vis[MAXN];           // dfs1 访问标记
```

```cpp
vector<int> order;          // 存第一遍 dfs 的后序离开顺序

int col[MAXN];              // col[u]：点 u 属于哪个强连通分量，编号从 1 开始
int cnt_scc;                // scc 总数量

// 第一遍 DFS：原图，记录后序（离开）顺序
void dfs1(int u)
{
    vis[u] = true;
    for (int v : g[u])
    {
        if (!vis[v])
        {
            dfs1(v);
        }
    }
    order.push_back(u); // 递归全部走完，离开 u 的时候压入 order
}

// 第二遍 DFS：反图，搜集同一个 SCC 内所有点，染色
void dfs2(int u)
{
    col[u] = cnt_scc;
    for (int v : gr[u])
    {
        if (col[v] == 0) // 还没有分配分量编号
        {
            dfs2(v);
        }
    }
}

// Kosaraju 主函数
// n：点的数量，点编号 1~n
void kosaraju(int n)
{
    // 初始化
    fill(vis, vis + n + 1, false);
    order.clear();
    fill(col, col + n + 1, 0);
    cnt_scc = 0;

    // 步骤 1：原图 DFS，得到后序序列 order
    for (int i = 1; i <= n; ++i)
    {
        if (!vis[i])
        {
            dfs1(i);
        }
    }

    // 步骤 2：逆序遍历 order，在反图上 DFS 求 SCC
    reverse(order.begin(), order.end());
    for (int u : order)
```

```
    {
        if (col[u] == 0) // 还没分到分量
        {
            cnt_scc++;
            dfs2(u);
        }
    }
}

// ========== 缩点构建 DAG ==========
vector<int> dag[MAXN];  // 缩点后的 DAG 邻接表

void build_dag(int n)
{
    // 清空 DAG
    for (int i = 1; i <= cnt_scc; i++) dag[i].clear();

    // 去重：对每个 a，存已经加过的 b，避免重边
    vector<unordered_set<int>> tmp(cnt_scc + 1);

    for (int u = 1; u <= n; u++)
    {
        for (int v : g[u])
        {
            int a = col[u];
            int b = col[v];
            if (a != b) // 不在同一个 scc，才加 DAG 边
            {
                if (!tmp[a].count(b))
                {
                    tmp[a].insert(b);
                    dag[a].push_back(b);
                }
            }
        }
    }
}

// ========== 使用示例 main ==========
/*
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    for(int i = 1; i <= m; i++)
    {
        int u, v;
        cin >> u >> v;
        g[u].push_back(v);    // 原图 u->v
        gr[v].push_back(u);   // 反图 v<-u 等价 gr[v].push_back(u)
    }
```

```
    kosaraju(n);        // 求 SCC
    build_dag(n);       // 缩点建 DAG

    cout << " 总 SCC 数目: " << cnt_scc << "\n";
    for(int i = 1; i <= n; i++)
    {
        cout << " 点" << i << " 属于分量: " << col[i] << "\n";
    }

    return 0;
}
*/
```

树

LCA

倍增法

```
// dfs，用来为 lca 算法做准备。接受两个参数：dfs 起始节点和它的父亲节点。
void dfs(int root, int fno) {
    // 初始化：第 2^0 = 1 个祖先就是它的父亲节点，dep 也比父亲节点多 1。
    fa[root][0] = fno;
    dep[root] = dep[fa[root][0]] + 1;
    // 初始化：其他的祖先节点：第 2^i 的祖先节点是第 2^(i-1) 的祖先节点的第
    // 2^(i-1) 的祖先节点。
    for (int i = 1; i < 31; ++i) {
        fa[root][i] = fa[fa[root][i - 1]][i - 1];
        cost[root][i] = cost[fa[root][i - 1]][i - 1] + cost[root][i - 1];
    }
    // 遍历子节点来进行 dfs。
    int sz = v[root].size();
    for (int i = 0; i < sz; ++i) {
        if (v[root][i] == fno) continue;
        cost[v[root][i]][0] = w[root][i];
        dfs(v[root][i], root);
    }
}

// lca。用倍增算法算取 x 和 y 的 lca 节点。
int lca(int x, int y) {
    // 令 y 比 x 深。
    if (dep[x] > dep[y]) swap(x, y);
    // 令 y 和 x 在一个深度。
    int tmp = dep[y] - dep[x], ans = 0;
    for (int j = 0; tmp; ++j, tmp >>= 1)
        if (tmp & 1) ans += cost[y][j], y = fa[y][j];
    // 如果这个时候 y = x，那么 x，y 就都是它们自己的祖先。
    if (y == x) return ans;
    // 不然的话，找到第一个不是它们祖先的两个点。
    for (int j = 30; j >= 0 && y != x; --j) {
        if (fa[x][j] != fa[y][j]) {
            ans += cost[x][j] + cost[y][j];
            x = fa[x][j];
            y = fa[y][j];
```

```
        }
    }
    // 返回结果。
    ans += cost[x][0] + cost[y][0];
    return ans;
}
```

利用欧拉序转化为 RMQ 问题

```
const int MAXN = 500005;
const int LOG  = 21;

int n, m, root;
vector<int> g[MAXN];

int euler[MAXN << 1];    // 欧拉序（存节点编号），长度 2n-1
int dep[MAXN];           // 深度
int first[MAXN];         // 每个节点第一次出现的位置
int tot;                 // 欧拉序长度

int st[LOG][MAXN << 1]; // ST 表，存节点编号（比较深度）
int lg[MAXN << 1];

void dfs(int u, int fa, int d) {
    dep[u] = d;
    first[u] = ++tot;
    euler[tot] = u;
    for (int v : g[u]) {
        if (v == fa) continue;
        dfs(v, u, d + 1);
        euler[++tot] = u;       // 回溯时再记录一次
    }
}

void buildST() {
    lg[1] = 0;
    for (int i = 2; i <= tot; ++i) lg[i] = lg[i >> 1] + 1;
    for (int i = 1; i <= tot; ++i) st[0][i] = euler[i];
    for (int k = 1; (1 << k) <= tot; ++k)
        for (int i = 1; i + (1 << k) - 1 <= tot; ++i) {
            int a = st[k - 1][i];
            int b = st[k - 1][i + (1 << (k - 1))];
            st[k][i] = dep[a] < dep[b] ? a : b;
        }
}

int lca(int u, int v) {
    int l = first[u], r = first[v];
    if (l > r) swap(l, r);
    int k = lg[r - l + 1];
    int a = st[k][l];
    int b = st[k][r - (1 << k) + 1];
    return dep[a] < dep[b] ? a : b;
}
```

计算每个节点作为根时，子节点的子树的大小（CF2241E）

```
vector<int> g[200010],son[200010],vis(200010),cnt(200010);
void dfs(int x){
    vis[x]=1;
    for(auto node:g[x]){
        if(vis[node]) continue;
        dfs(node);
        cnt[x]+=cnt[node];
        son[x].push_back(cnt[node]);
    }
    if(x!=1) son[x].push_back(n-cnt[x]);
}
```

网络流

Dinic

```
struct MF {
    struct edge {
        int v, nxt, cap, flow;
    } e[N];

    int fir[N], cnt = 0;

    int n, S, T;
    int maxflow = 0;
    int dep[N], cur[N];

    void init() {
        memset(fir, -1, sizeof fir);
        cnt = 0;
    }

    void addedge(int u, int v, int w) {
        e[cnt] = {v, fir[u], w, 0};
        fir[u] = cnt++;
        e[cnt] = {u, fir[v], 0, 0};
        fir[v] = cnt++;
    }

    bool bfs() {
        queue<int> q;
        memset(dep, 0, sizeof(int) * (n + 1));

        dep[S] = 1;
        q.push(S);
        while (q.size()) {
            int u = q.front();
            q.pop();
            for (int i = fir[u]; ~i; i = e[i].nxt) {
                int v = e[i].v;
                if ((!dep[v]) && (e[i].cap > e[i].flow)) {
                    dep[v] = dep[u] + 1;
                    q.push(v);
```

```
                }
            }
        }
        return dep[T];
    }

    int dfs(int u, int flow) {
        if ((u == T) || (!flow)) return flow;

        int ret = 0;
        for (int& i = cur[u]; ~i; i = e[i].nxt) {
            int v = e[i].v, d;
            if ((dep[v] == dep[u] + 1) &&
                (d = dfs(v, min(flow - ret, e[i].cap - e[i].flow)))) {
                ret += d;
                e[i].flow += d;
                e[i ^ 1].flow -= d;
                if (ret == flow) return ret;
            }
        }
        return ret;
    }

    void dinic() {
        while (bfs()) {
            memcpy(cur, fir, sizeof(int) * (n + 1));
            maxflow += dfs(S, INF);
        }
    }
} mf;
```

费用流

```
#include <algorithm>
#include <cstdio>
#include <cstring>
#include <queue>

constexpr int N = 5e3 + 5, M = 1e5 + 5;
constexpr int INF = 0x3f3f3f3f;
int n, m, tot = 1, lnk[N], cur[N], ter[M], nxt[M], cap[M], cost[M], dis[N], ret;
bool vis[N];

void add(int u, int v, int w, int c) {
    ter[++tot] = v, nxt[tot] = lnk[u], lnk[u] = tot, cap[tot] = w, cost[tot] = c;
}

void addedge(int u, int v, int w, int c) { add(u, v, w, c), add(v, u, 0, -c); }

bool spfa(int s, int t) {
    memset(dis, 0x3f, sizeof(dis));
    memcpy(cur, lnk, sizeof(lnk));
    std::queue<int> q;
    q.push(s), dis[s] = 0, vis[s] = true;
```

```
    while (!q.empty()) {
        int u = q.front();
        q.pop(), vis[u] = false;
        for (int i = lnk[u]; i; i = nxt[i]) {
            int v = ter[i];
            if (cap[i] && dis[v] > dis[u] + cost[i]) {
                dis[v] = dis[u] + cost[i];
                if (!vis[v]) q.push(v), vis[v] = true;
            }
        }
    }
    return dis[t] != INF;
}

int dfs(int u, int t, int flow) {
    if (u == t) return flow;
    vis[u] = true;
    int ans = 0;
    for (int &i = cur[u]; i && ans < flow; i = nxt[i]) {
        int v = ter[i];
        if (!vis[v] && cap[i] && dis[v] == dis[u] + cost[i]) {
            int x = dfs(v, t, min(cap[i], flow - ans));
            if (x) ret += x * cost[i], cap[i] -= x, cap[i ^ 1] += x, ans += x;
        }
    }
    vis[u] = false;
    return ans;
}

int mcmf(int s, int t) {
    int ans = 0;
    while (spfa(s, t)) {
        int x;
        while ((x = dfs(s, t, INF))) ans += x;
    }
    return ans;
}

int main() {
    int s, t;
    scanf("%d%d%d%d", &n, &m, &s, &t);
    while (m--) {
        int u, v, w, c;
        scanf("%d%d%d%d", &u, &v, &w, &c);
        addedge(u, v, w, c);
    }
    int ans = mcmf(s, t);
    printf("%d %d\n", ans, ret);
    return 0;
}
```

字符串

KMP

```cpp
#include <iostream>
#include <vector>
#include <string>

using namespace std;

// 构建 next 数组
vector<int> buildNext(const string &pattern) {
    int m = pattern.size();
    vector<int> next(m, 0);
    int j = 0; // 前缀末尾指针

    for (int i = 1; i < m; ++i) {
        while (j > 0 && pattern[i] != pattern[j]) {
            j = next[j - 1]; // 回退到上一个可能的前缀位置
        }
        if (pattern[i] == pattern[j]) {
            ++j;
        }
        next[i] = j;
    }
    return next;
}

// KMP 主函数
vector<int> kmpSearch(const string &text, const string &pattern) {
    int n = text.size(), m = pattern.size();
    vector<int> next = buildNext(pattern);
    vector<int> result; // 存储匹配起始位置
    int j = 0;          // 模式串指针

    for (int i = 0; i < n; ++i) {
        while (j > 0 && text[i] != pattern[j]) {
            j = next[j - 1]; // 回退
        }
        if (text[i] == pattern[j]) {
            ++j;
        }
        if (j == m) { // 完全匹配
            result.push_back(i - m + 1); // 记录匹配起始位置
            j = next[j - 1];             // 寻找下一个可能匹配
        }
    }

    return result;
}

int main() {
    string text = "ababcabcacbab";
    string pattern = "abcac";
```

```
    vector<int> matches = kmpSearch(text, pattern);

    cout << " 匹配位置: ";
    for (int pos : matches) {
        cout << pos << " ";
    }
    cout << endl;

    return 0;
}
```

Z 函数

```
vector<int> z_function(string s) {
  int n = (int)s.length();
  vector<int> z(n);
  for (int i = 1, l = 0, r = 0; i < n; ++i) {
    if (i <= r && z[i - l] < r - i + 1) {
      z[i] = z[i - l];
    } else {
      z[i] = max(0, r - i + 1);
      while (i + z[i] < n && s[z[i]] == s[i + z[i]]) ++z[i];
    }
    if (i + z[i] - 1 > r) l = i, r = i + z[i] - 1;
  }
  return z;
}
```

Manacher

```
vector<int> d1,d2;
void manacher_d1d2(const string &s) {
    int n = s.size();
    d1.assign(n, 0);
    d2.assign(n, 0);

    // 算 d1 奇数
    for (int i = 0, l = 0, r = -1; i < n; i++) {
        int k = (i > r) ? 1 : min(d1[l + r - i], r - i);
        while (i - k >= 0 && i + k < n && s[i - k] == s[i + k]) k++;
        d1[i] = --k;
        if (i + d1[i] > r) {
            l = i - d1[i];
            r = i + d1[i];
        }
    }

    // 算 d2 偶数
    for (int i = 0, l = 0, r = -1; i < n; i++) {
        int k = (i > r) ? 0 : min(d2[l + r - i + 1], r - i + 1);
        while (i - k - 1 >= 0 && i + k < n && s[i - k - 1] == s[i + k]) k++;
        d2[i] = k;
        if (i + d2[i] - 1 > r) {
```

```
            l = i - d2[i];
            r = i + d2[i] - 1;
        }
    }
}
```

字典树

```
struct trie {
    int nex[100000][26], cnt;
    bool exist[100000];  // 该结点结尾的字符串是否存在
    void init() {
        cnt = 0;
        memset(nex, 0, sizeof(nex));
        memset(exist, 0, sizeof(exist));
      }
      void insert(char *s, int l) {  // 插入字符串
        int p = 0;
        for (int i = 0; i < l; i++) {
              int c = s[i] - 'a';
          if (!nex[p][c]) nex[p][c] = ++cnt;  // 如果没有，就添加结点
          p = nex[p][c];
    }
    exist[p] = true;
}

      bool find(char *s, int l) {  // 查找字符串
        int p = 0;
        for (int i = 0; i < l; i++) {
              int c = s[i] - 'a';
            if (!nex[p][c]) return 0;
            p = nex[p][c];
        }
        return exist[p];
      }
};
```

AC 自动机

```
#include <iostream>
#include <queue>
#include <cstring>
using namespace std;

const int ALPHABET_SIZE = 26; // 假设只处理小写字母
const int MAX_NODES = 100005; // 最大节点数，根据需求调整

struct AC_Automaton {
    int trie[MAX_NODES][ALPHABET_SIZE]; // Trie 树
    int fail[MAX_NODES];                // 失配指针
    int output[MAX_NODES];              // 输出标记
    int nodeCount;                      // 当前节点计数

    // 初始化
```

```
void init() {
    memset(trie, 0, sizeof(trie));
    memset(fail, 0, sizeof(fail));
    memset(output, 0, sizeof(output));
    nodeCount = 1; // 根节点为 0，从 1 开始分配新节点
}

// 插入模式串
void insert(const string &word) {
    int currentNode = 0;
    for (char c : word) {
        int index = c - 'a';
        if (!trie[currentNode][index]) {
            trie[currentNode][index] = nodeCount++;
        }
        currentNode = trie[currentNode][index];
    }
    output[currentNode]++; // 标记模式串结尾
}

// 构建失配指针
void build() {
    queue<int> q;
    for (int i = 0; i < ALPHABET_SIZE; ++i) {
        if (trie[0][i]) {
            fail[trie[0][i]] = 0;
            q.push(trie[0][i]);
        }
    }

    while (!q.empty()) {
        int currentNode = q.front();
        q.pop();

        for (int i = 0; i < ALPHABET_SIZE; ++i) {
            if (trie[currentNode][i]) {
                int nextNode = trie[currentNode][i];
                int failure = fail[currentNode];

                while (failure && !trie[failure][i]) {
                    failure = fail[failure];
                }
                fail[nextNode] = trie[failure][i];
                q.push(nextNode);
            } else {
                trie[currentNode][i] = trie[fail[currentNode]][i];
            }
        }
    }
}

// 匹配文本
int search(const string &text) {
    int currentNode = 0;
```

```
        int matchCount = 0;

        for (char c : text) {
            int index = c - 'a';
            currentNode = trie[currentNode][index];

            for (int temp = currentNode; temp; temp = fail[temp]) {
                matchCount += output[temp];
                output[temp] = 0; // 避免重复计数
            }
        }
        return matchCount;
    }
};

int main() {
    AC_Automaton ac;
    ac.init();

    int n;
    cout << " 输入模式串数量: ";
    cin >> n;

    cout << " 输入模式串:" << endl;
    for (int i = 0; i < n; ++i) {
        string pattern;
        cin >> pattern;
        ac.insert(pattern);
    }

    ac.build();

    string text;
    cout << " 输入待匹配文本: ";
    cin >> text;

    int result = ac.search(text);
    cout << " 匹配到的模式串总数: " << result << endl;

    return 0;
}
```

动态规划

线性 DP

最长上升子序列

```
int lengthOfLIS(vector<int>& nums) {
    vector<int> tails;
    for(int x : nums){
        auto it = lower_bound(tails.begin(), tails.end(), x);
        if(it == tails.end()) tails.push_back(x);
        else *it = x;
```

```
    }
    return tails.size();
}
```

状压 DP

状压 DP 求拓扑序数量

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n, m;
    cin >> n >> m;
    vector<int> pre(n, 0); // pre[u]: u 所有前驱的掩码
    for (int i = 0; i < m; i++) {
        int u, v;
        cin >> u >> v;
        u--; v--;
        pre[v] |= (1 << u);
    }

    vector<int> dp(1 << n, 0);
    dp[0] = 1;

    for (int mask = 0; mask < (1 << n); mask++) {
        if (dp[mask] == 0) continue;
        // 尝试选下一个点 u
        for (int u = 0; u < n; u++) {
            if (!(mask & (1 << u)) && ((pre[u] & mask) == pre[u])) {
                dp[mask | (1 << u)] += dp[mask];
            }
        }
    }
    cout << dp[(1 << n) - 1] << endl;
    return 0;
}
```